NGN220 – First Python Scripting

(The following is derived from https://docs.python.org/3.8/tutorial/index.html, other Python instruction sources, and my editing).

Python – an explanation
The Python programming language is named after the BBC show "Monty Python's Flying Circus" and has nothing to do with reptiles. Making references to Monty Python skits in documentation is not only allowed, it is encouraged!

Exploring Python
Python can be accessed from multiple locations. The most "basic" way is through the *command prompt (kommandotolken)*. If you work on a Mac, this is *Terminal*! But this is not the easiest way to work with Python.

Why should you learn Python?
You will find that many programs integrate Python, such as QGIS, ArcMap, and Matlab. You can do a lot of boring work more efficiently. It can be a skill that gets you a job more easily, and also save you a lot of time and effort.

Once you have an idea of the appropriate building blocks needed to solve a specific problem, then you can worry about more details. Particularly at the beginning, you are not likely to have all the exact Python syntax for the parts of your solution nailed down! It is not important to remember it precisely, but it is important to know how to find a clear explanation efficiently

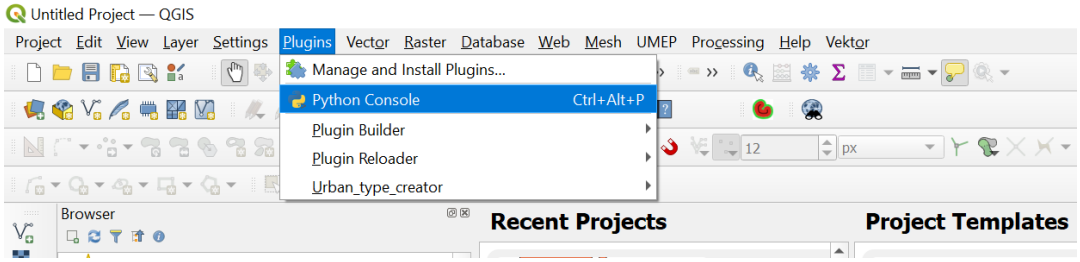
Some notes: It is good practice to avoid folder-names that contain any blanks!

DON'T TO *"C:/data/Johan Svensson/Python"*

DO *"C:/data/Johan_Svensson/Python"*

Start with Python
Let's try some simple Python commands.

Start the *Command Prompt (kommandotolken)*. Type python and hit enter. If you have Python installed on your computer, you should now be in a Python environment! If not, open QGIS and use the Python console there.



Using Python as a Calculator
You can type an expression in the Python console and it will write the value. Expression syntax is straightforward: the mathematical operators +, -, *, (), and / work similar to most other languages such as C++. To execute an expression press enter!

For example:

>>> 2 + 2

>>> 50 - 5*6

>>> (50 - 5*6) / 4

>>> 8 / 5

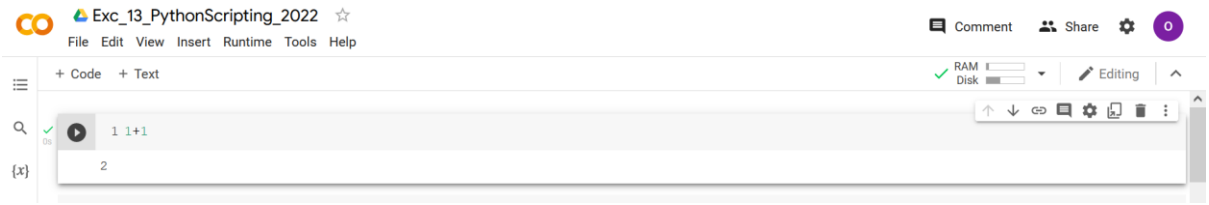
Note! The >>> above is a way of indicating that we are in a Python environment. You shall not write >>> ! It is also good in this exercise because it makes It more difficult for you to copy and paste…

Try the examples above, or, of course, your own. If you get an error, try again!

Running Python in the command prompt is ok for small operations, but not for longer scripts. It is also not that user friendly…

Now we will move into a more user-friendly Python interpreter, a Jupyter Notebook.

Start a new Jupyter Notebook in Google Colab
Create a new Jupyter Notebook in Google Colab. Change the name to something that you think is reasonable.
Jupyter Notebook allow for executing Python expressions in *cells.* Below is an example of executing a Python command in a cell. Click the small arrow to execute the cell, or simply go *ctrl+enter*.



To add a cell, click + Code.

Start a new Jupyter Notebook from OSGeo Shell
Navigate to a directory where you want to create your file (cd, mkdir, etc., see https://www.computerhope.com/msdos.htm for commands).
When you are in the directory where you want to create your file, you write jupyter notebook and press enter. Jupyter Notebook should now start in your web browser. Press New (upper right corner) and choose “Python 3 (ipykernel)”.
To add line numbers, go to View -> Show Line Numbers (ctrl+L should also work).

Types in Python
Some types are int (integer) or float (decimal). In the examples above, the integer numbers (e.g., 2, 4, 20) have type int, the ones with a fractional part (e.g., 5.0, 1.6) have type float. We will learn more about numeric types later in the tutorial.

PLEASE READ THIS! When you see line breaks in code below, this indicate in this tutorial to create a new cell! So for instance

>>> 1+1 (This is one cell!)

>>> 9*2 (This is another cell!)

Division (/) always returns a float. To do mathematical division that rounds down to nearest integer (i.e., so-called “floor division”) and get an integer result (discarding any fractional result) you can use the // operator; to calculate the remainder you can use %:

```
>>> 17 / 3
>>> # division returns a float

>>> 17 // 3
>>> # floor division discards the fractional part

>>> 17 % 3
>>> # the % operator returns the remainder of the division

>>> 5 * 3 + 2
>>> # result * divisor + remainder
```

Try the commands above in individual cells!

Commenting your script
You can see from the above script that there is text that gives comments about the script. It has a # sign in front of the text which tells Python that this is a comment, and it is not a command that the computer should execute. It basically ignores it. It is a very good way to remind yourself, or another person who might use your script, what you have done. Good commenting is a VERY GOOD habit to learn early when you are programming. (Although the example above is maybe a bit too obvious comments – we will look at more commenting examples later).
You can put your comments anywhere – the following also works. Python ignores everything in the line that comes after the # sign.

```
>>> 17 // 3 # floor division discards the fractional part
```

With Python, it is possible to use the ** operator to calculate powers
```
>>> 2 ** 3
```

Variables
Using variables is a powerful way to work with any type of programming and scripting. Now we are moving towards really making use of the power of Python and scripting!

A variable is something that you name that represent/is equal to something you assign it to be. It can be *x, y, height, meters_above_sea_level* or anything else that is reasonable to the task.

NOTE! Python is case-sensitive, so x and X is not the same, and height and Height is not the same!

The equal sign (=) is used to assign a value to a variable. A Python variable is a reserved memory location to store values. In other words, a variable in a Python program gives data to the computer for processing. Every value in Python has a datatype. Different data types in Python are Numbers, List, Tuple, Strings, Dictionary, etc.

Note that the variable below can be called “width” or “x” or “my_data”. Note that it is one word and has the _ in it. This is typical practice and helps avoid the program reading the space as the end of the command or a mistake in the command. Afterwards, no result is displayed before the next interactive prompt.

```
>>> width = 20
>>> # we create a variable called “width”, or whatever we want to call
it
>>> height = 5 * 9
>>> width * height

>>> # Alternatively
>>> x = 20
>>> x * height

>>> # try the following too
>>> my data = 29
>>> my_data = 30

>>> # note you can assign several variable names and values at once as
follows
```

```
>>> a, b = 0, 1
>>> # test it by checking the values of a and b, separately using a pri
nt() command
>>> print(a)
>>> print(b)
```

Strings

Besides numbers, Python can also manipulate text, which is referred to as “strings”, which can be expressed in several ways. They can be enclosed in 'single quotes' or "double quotes". The symbol \ can be used to escape quotes.

Try the following...

```
>>> "Geo"
```

```
>>> 'Isn\'t it GEO?'
```

```
>>> '"Isn\'t it?", GEO?'
```

note that using " gives a different result than regular text quote marks “ ”, which Python won’t recognize. If you try to copy/paste code that has “ ”,Python will give you an error message.

Besides this, you can use the print() function. The symbol \n means newline.

```
>>> print('"Isn\'t it GEO?," they said.')
>>> print('Isn\'t it?')
```

The print() function writes the value of the arguments it is given. It differs from just writing the expression you want to write (as we did earlier in the calculator examples) in the way it handles multiple arguments, floating point quantities, and strings. Strings are printed without quotes, and a space is inserted between items, so you can format things nicely, like this:

```
>>> z = 256*256
>>> print('The value of z is', z)
```

Strings can be concatenated (glued together) with the + operator, and repeated with *:

```
>>> word1 = 3 * 'GEO' + 'vetenskap'
```

You see you don’t get any value returned in the cell, but you can type the name of the variable to get the value.

```
>>> word1
```

The built-in function len( ) returns the length of a string

```
>>> len(word1)
```

Two or more string literals (i.e., the ones enclosed between quotes) next to each other are automatically concatenated.

```
>>> 'Py' 'thon'
```

Strings can be indexed (subscripted), with the first character having index 0. There is no separate character type; a character is simply a string of size one. The variable here is word.

```
>>> word = 'Python'
>>> word[0] # character in position 0
```

Try changing [0] to get the letter *o*

In addition to indexing, slicing is also supported. Slicing allows you to cut-up data into pieces. While indexing is used to obtain individual characters, slicing allows you to obtain substrings:

```
>>> word[0:2] # characters from position 0 (included) to 2 (excluded)

>>> word[2:6] # characters from position 2 (included) to 6 (excluded)
```

Lists

Python knows a number of compound data types, used to group together other values. One of the most versatile is the list, which can be written as a list of comma-separated values (items) between square brackets. Lists might contain items of different types, but usually the items all have the same type.

```
>>> squares = [1, 4, 9, 16, 25]
>>> squares[0] # indexing returns the item

>>> squares[0:3] # indexing returns the item
```

Unlike strings, which you cannot edit (they are “immutable”), lists are a mutable type, i.e. it is possible to change their content.

```
>>> cubes = [1, 8, 27, 65, 125] # something's wrong here. 65 should be
64.
>>> cubes[3] = 64 # replace the wrong value
>>> cubes
```

Or...

```
>>> letters = ['a', 'b', 'c', 'd']
>>> letters
```

It is possible to nest lists (create lists containing other lists), for example:

```
>>> a = ['a', 'b', 'c']
>>> n = [1, 2, 3]
>>> # now we can create a variable x
>>> # then we put together list a and n, and assign it to x
>>> # see what happens when we do that
>>> x = [a, n]
>>> x
```

Tuples
There is also another standard sequence data type: *the tuple*. Tuples are immutable, meaning that once created you cannot change any variables within the tuple as you can do with a list. Tuples are created using comma separation, with and without soft brackets (()).

```
>>> p = ('a', 'b', 'c')
>>> q = 6, 4, 9

>>> # Try now to change the first value q to number 3 and see what
happens
>>> q[0] = 3
```

Dictionaries
Another useful data type built into Python is the *dictionary*. Unlike sequences (tuples and lists), which are indexed by a range of numbers, dictionaries are indexed by *keys*, which can be any immutable type; strings and numbers can always be keys.

It is best to think of a dictionary as a set of *key: value* pairs, with the requirement that the keys are unique (within one dictionary). A pair of braces creates an empty dictionary: {}. Placing a comma-separated list of key:value pairs within the braces adds initial key:value pairs to the dictionary; this is also the way dictionaries are written on output.

```
>>> tel = {'jack': 4098, 'sape': 4139}
>>> tel['guido'] = 4127
>>> tel

>>> tel['jack']

>>> del tel['sape']
>>> tel['irv'] = 4127
>>> tel

>>> tel.keys()
```

The last line make use of a built-in function (keys() found in all dictionaries. As Python is a object-oriented programming language meaning that everything written in Python is actually objects which have certain methods (functions) inherited in them. For example, a dictionary has the built in function (method) keys() that gives you all the keys currently in the dictionary.

Extending your Python
Python is an open-source programming language which means that user and contributors can create code libraries/packages that can be used of others. There are thousands of different libraries that can be installed and used on your system. In Colab there are already a number useful libraries installed suck as numpy, scipy, pandas. A list of useful packages can be found here (https://flexiple.com/python/python-libraries).

To access an installed library, we need to import it. Lets try an example using numpy. Numpy is alibrary used for numerical computing, especially useful if you work with arrays and matrices:

```
>>> import numpy as np
>>> np.pi
3.141592653589793
```

Steps Towards Programming

Flow controls

Of course, we can use Python for more complicated tasks than adding two and two together. For instance, we can write an initial sub-sequence of the Fibonacci series
(See https://en.wikipedia.org/wiki/Fibonacci_number).

Note that some of the commands below are indented. This happens not only to keep order, but the program is expecting these indents when it is waiting for other information. To indent, press tab-key




```
>>> a, b = 0, 1
>>> while a < 10:
>>>  print(a)
>>>  a = a + b
```

Run the cell and examine the output. What happen if you change while a < 10 to something else?

Take a moment to understand why you got this result.

The while loop

- The while loop executes as long as the condition (here the condition is “a < 10”) remains true. The program “evaluates” the condition, and if the condition is false, this results in assigning “0” as the value. If the condition is true, the value assigned is any integer not equal to 0 (a non-zero integer).
- The condition may also be a string or list value, in fact any sequence; anything with a non-zero length is evaluated as “true”, empty sequences are “false”. The test used in the example is a simple comparison. The standard comparison operators are written the same as in C: < (less than), > (greater than), == (equal to), <= (less than or equal to), >= (greater than or equal to) and != (not equal to).
- The body of the loop is indented: indentation is Python’s way of grouping statements.

If statements

If statements are a type of loop that can run under different conditions. A note, **don’t just copy/paste the whole command text below** - you need to give input at line 2, also the indentation will be important. So type these in by hand. You can read the interpretation of the if statement in the text after it.

A simple if statement with an input provided by the user

```
>>> ph = float(input("What is the pH of your sample? "))
>>> if ph > 7.5:
>>>  print("Your sample is acidic.")
>>> elif ph == 7:
```

```
>>>  print ("Your sample is neutral with a pH of 7")
>>> else:
>>>  print("Your sample is below the acidic threshold.")
```

What the first line does is it creates variable “ph” and assigns the value from the input given at the next prompt, and makes it an integer (int()).

The next line, “`if` ph > 7.5”, evaluates the statement. The colon at the end of the line means it is waiting for input from the next line. If the statement evaluates as being “true”, it will perform the next line and print “Your sample is acidic”. But what if it is less than 7.5? Then you can have an “else if” statement which is represented by “`elif`”. You can have any number of “else if” statements, but then you can end it with an “`else`” (this is optional), which takes care of all other cases not already handled.

Errors: (yikes!)

Errors will appear **a lot** but it is nothing to worry about. You learn through dealing with errors all the time! The error text will help you locate and *debug* your code: like this

```
[52]  1 #A simple if statement with an input
      2 ph = float(input("What is the pH of your sample? "))
      3 if ph > 7.5:
      4   print("Your sample is acidic.")
      5  elif ph == 7:
      6   print ("Your sample is neutral with a pH of 7")
      7 else:
      8   print("Your sample is below the acidic threshold.")
      9
```

```
  File "<ipython-input-52-e56ca6c37567>", line 5
    elif ph == 7:
                ^
IndentationError: unindent does not match any outer indentation level
```

The error text tells me that I have an error in line 5.

Did you get a Syntax Error like this?
SyntaxError: inconsistent use of tabs and spaces in indentation
Then you just need to start the “if ph > 7.5” loop again, and use consistent tabs for indentation. You need to use the Backspace to get back to the left hand side of cell.
SyntaxError: invalid syntax
Then somewhere you have used something in the command that is not correct. You will need to look for it, modify, and re-run. This is part of “debugging” the code.
SyntaxError: invalid character in identifier
You may have used a “or” instead of "

For more on errors, look here https://docs.python.org/3.9/tutorial/errors.html#

For Loops

Python structure of "for statements" is different than other programming languages, such as C++. The basic structure is

```
for <var> in <iterable>:
    <statement(s)>
```

An <iterable> is a collection of objects—for example, a list. The <statement(s)> in the loop body are denoted by indentation, as with all Python control structures, and are executed once for each item in <iterable>. The loop variable <var> takes on the value of the next element in <iterable> each time through the loop.

Do this example

```
>>> a = [1, 2, 3]
>>> for i in a:
>>>   print(i)
```

In this example, <iterable> is the list a, and <var> is the variable i. Each time through the loop, i takes on a successive item in a, so print() displays the values 1, 2 and 3, respectively, for every value in list a, until it is complete.

There is much more information about iterables, but we will leave it there for now. The example below exemplifies how to loop through a list.

```
>>> a = [1,2,3,4]

>>> total = 0
>>> for i in a:
>>>   print('i:',i)
>>>   total = total + i
>>>   print('total:', total)

It is also possible to convert integers to strings and use in loops.
Try the following lines and see what happen

>>> a = [1,2,3,4]
>>> total = str(0)
>>> for i in a:
>>>   print('i:',str(i))
>>>   total = total + str(i)
>>>   print('total:', total)
```

Range function

If you do need to iterate over a sequence of numbers, the built-in function range() comes in handy. It generates arithmetic progressions.

```
>>> # note that range includes the first number as the lower limit, but
 not the last number
>>> for i in range(5,10):
>>>   print(i)
```

```
>>> # In this example, you can even give the range, then the third numb
er indicates the increment to use
>>> for i in range(0, 20, 3):
>>>   print (i)
```

Or try these

```
>>> sum(range(5,10))
```

```
>>> list(range(5,10))
```

Break or Continue in your Loops
Sometimes you may want some information from the loop, and not just the end result. For this, you can use Break or Continue. The break statement breaks out of the innermost enclosing for or while loop. Loop statements may have an else clause; it is executed when the loop terminates through exhaustion of the iterable (in the for loop) or when the condition becomes false (in the while loop), but not when the loop is terminated by a break statement. This is exemplified by the following loop, which searches for even numbers. The continue statement continues with the next iteration of the loop.

Example of For Loop with Continue

```
>>> for num in range(2, 13):
>>>   if num % 2 == 0:
>>>     print("Found an even number", num)
>>>     continue
>>>   print(num,"is not an even number")
```

Now, clear your variable num using the delete function. Or else the following function will not work

```
>>> del(num)
```

Example of For Loop with Break

```
>>> for num in range(2, 13):
>>>   for x in range(2, num):
>>>     if num % x == 0:
>>>       print(num, 'equals', x, '*', num//x)
>>>       break
>>>   else:
>>>     print(num,"is a prime number")
```

What should you be getting for a result? See on the last page for the answer.

Saving your Python code as a script
Here is one very good reason to learn and use scripting/programming:

You can save your code and use it again. You can change one little variable and save yourself a lot of work. You can pass your code on to others.

As you are working in Google Colab, your script is saved in the cloud on your google-account. You can share it to others similar to how you share a google-doc or google-spreadsheet. But you can also export it as a file.

Go to file→download→ here you can download as either a .ipynb (Jupyter Notebook file) or a .py file which is a pure python file.

The Assignment (hand-in)

Start a NEW Jupyter Notebook in Google Colab and write your own small program based on what you have learned so far. You can be as ambitious (or not) as you want, **as long as it works, and it is longer than 5 lines of code.**

Save it as a *.py file, and then zip or compress your *.py file (using 7-zip or similar) and turn in the zipped/compressed .py file on Canvas. The reason to zip it is so the computer doesn't think it is hostile code that someone is sending.

To clarify: Hand in ONLY the NEW Jupyter Notebook containing your own small program to canvas.

Resources
There are many, many resources on the Internet, and on YouTube. Here are a few:
https://www.python.org

https://automatetheboringstuff.com

https://www.fullstackpython.com/best-python-resources.html

https://www.codeconquest.com/blog/the-50-best-websites-to-learn-python

http://anh.cs.luc.edu/handsonPythonTutorial/index.html

https://realpython.com/python-youtube-channels

https://www.youtube.com/watch?v=Z1Yd7upQsXY

https://www.youtube.com/watch?v=wsSEKm-rU6U

https://www.youtube.com/watch?v=rfscVS0vtbw

Answers

FOR LOOP WITH BREAK

```
for num in range(2, 13):
  for x in range(2, num):
    if num % x == 0:
      print(num, 'equals', x, '*', num//x)
      break
  else:
    print(num,"is a prime number")
```

```
2 is a prime number
3 is a prime number
4 equals 2 * 2
5 is a prime number
6 equals 2 * 3
7 is a prime number
8 equals 2 * 4
9 equals 3 * 3
10 equals 2 * 5
11 is a prime number
12 equals 2 * 6
```

FOR LOOP WITH CONTINUE

```
for num in range(2, 13):
  if num % 2 == 0:
    print("Found an even number", num)
    continue
  print(num,"is not an even number")
```

```
Found an even number 2
3 is not an even number
Found an even number 4
5 is not an even number
Found an even number 6
7 is not an even number
Found an even number 8
9 is not an even number
Found an even number 10
11 is not an even number
Found an even number 12
```